Today 9:31 AM

Lab_4_VibeCoding_Student_handout....
PDF

Donkey Kong Repair Lab

This project is a single-file Donkey Kong-lite clone using **Pygame**. It introduces students to sloped-platform physics, ladder climbing, and gravity using a small, readable object-oriented codebase.

What's Provided

A working Donkey Kong-lite game with:

- A player-controlled climber with jumping and gravity that follows sloped platforms
- Ladders connecting each platform level that the player can climb
- Barrels that roll down sloped platforms and occasionally descend a ladder to the level below
- Lives, scoring, and a win condition for reaching the princess

It has **one deliberate bug** and **three optional features** left as empty functions. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install pygame
```

3. Run the game:

```
python game.py
```

Controls: Left/Right to move, Space to jump, Up/Down to grab a ladder while standing near it, `R` to reset.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the ladder-descent bug

> Barrels are supposed to take a ladder down to the next platform only occasionally, so a few keep rolling across the top platform while others trickle down toward the player. In the current build, almost every barrel immediately takes the first ladder it reaches, so the lower platforms flood with barrels and the top platform empties out fast. Find the probability check that controls this in the `Barrel` class and correct it so barrels descend roughly 3 times out of 10, not roughly 7 times out of 10.

Task 2: Implement `theme_color(score)`

> Called once per frame in `draw_scene` as `screen.fill(theme_color(score) or BG)`, where `BG` is the default navy background `(15, 15, 25)`. It receives the player's current integer score and should return an `(r, g, b)` tuple to use as the background color, or `None` to keep the default. Idea: shift toward a warmer color as the score climbs, or briefly flash a color right after a big score gain.

Task 3: Implement `on_barrel_jumped(player, barrel)`

> Called from the main loop exactly once, on the first frame the player is airborne directly above a barrel — right after the 100-point jump bonus is awarded for that barrel. It receives the `Player` and the `Barrel` that was jumped. Its return value is ignored; it exists purely for side effects. Idea: spawn a short-lived floating "+100" label at the barrel's position, or play a sound.

Task 4: Implement `score_multiplier(score)`

> Called right before the barrel-jump bonus is added, as `score += int(100 * (score_multiplier(score) or 1))`. It receives the score *before* the bonus is added and should return a numeric multiplier, or `None` for the default 1x. Idea: return `2` once the score passes some threshold, to reward late-game play.

Expected Behavior

- The player follows the slope of each platform instead of floating above it or sinking into it
- The player can only climb a ladder while standing near that ladder's x-position, on the platform range it connects
- Barrels roll down slopes and occasionally (about 30% of the time) descend a ladder instead of continuing to roll along the platform
- Barrels never fall through platforms
- Reaching the princess ends the game with a win message; running out of lives (from barrel collisions) ends it with a game-over message

---

Folder Structure

```
donkey_kong/
├── game.py
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history
- https://github.com/SETAPESU26/30_donkeykong

1.fork,2.clone the forked repo,3.video,4.make changes and commit and push it to my repo,5.video after all the changes
deliverables-on the repo that you have submitted lab 1-3 ,2 videos ,llm chat export ,GitHub repo link
-deliverables

Okay, let's do your **Lab 4: Vibe Coding — Donkey Kong Repair Lab** step by step, from forking the repository to submitting everything. I'll keep the commands beginner-friendly so you can follow them on your MacBook.

According to your handout, this is an individual assignment. You need to record gameplay before and after your changes, fix the bug, implement all three optional features, push your work to your own GitHub repository, and keep separate commits for each task. Lab_4_VibeCoding_Student_handout.pdf

Your complete workflow

1. Fork and clone the repository
Create your own copy of the assigned Donkey Kong project.

2. Record the BEFORE video
Capture the original game and its barrel-descent bug before editing anything.

3. Fix and implement the four tasks
One bug fix and three features, with a separate Git commit for each task.

4. Record the AFTER video
Show the corrected barrel behavior and the new features working.

5. Push and submit
Upload the updated code, both videos, and your exported AI chat history to your Lab repository.

Step 1: Fork the repository

1. Open the assigned repository: 30_donkeykong on GitHub ↗.
2. Sign in to **your own GitHub account**.
3. Click **Fork** at the top-right.
4. Click **Create fork**.

Your repository should look like:

```
https://github.com/YOUR-USERNAME/30_donkeykong
```

Replace `YOUR-USERNAME` with your actual GitHub username. Make sure you fork it to your account, not your friend's account.

Step 2: Clone your fork on your MacBook

Open Terminal and run these commands one by one.

First, go to your Desktop:

```bash
cd ~/Desktop
```

Clone your own fork — replace the username with yours:

```bash
git clone https://github.com/YOUR-USERNAME/30_donkeykong.git
```

Enter the project folder:

```bash
cd 30_donkeykong
```

Install Pygame:

```bash
python3 -m pip install pygame
```

Check the files:

```bash
ls
```

You should see `game.py` and `README.md`, or the files provided by the repository.

Step 3: Run the original game and record BEFORE

Run:

```bash
python3 game.py
```

If it opens successfully, play for about 10 seconds and record your screen. Show the original barrel behavior before changing the code.

On your Mac, you can press **Shift + Command + 5** to open the screen-recording controls.

Save the recording as:

`before.mp4`

If the game doesn't open, send me the full Terminal error before continuing.

Step 4: Fix the bug and add the features

We'll complete four tasks, one at a time, and commit each separately.

Task 1 — Barrel probability

Find the probability condition in the `Barrel` class. Correct it so the descent chance is approximately 30%, rather than 70%.

Task 2 — Background color

Implement `theme_color(score)` so the background changes gradually as the player's score increases.

Task 3 — Jump notification

Implement `on_barrel_jumped(player, barrel)` to display a temporary `+100` message when the player jumps over a barrel.

Task 4 — Score multiplier

Implement `score_multiplier(score)` so the player can earn double points after a chosen score threshold.

Important: I need to see your actual `game.py` before giving you exact code changes. That way, we can fit the features into the existing game without breaking its physics or collision logic.

After each task, we'll test it and make a separate commit.

Example commit commands:

```bash
git add game.py
git commit -m "Fix barrel ladder descent probability"
```

Use a separate commit message for each of the other three tasks as well.

Step 5: Record AFTER

Once all four tasks work:

1. Run `python3 game.py`.
2. Show the corrected barrel behavior.
3. Show the background color changing as the score increases.
4. Jump over a barrel to show `+100`.
5. Show the multiplier working, if you can reach the threshold.
6. Record approximately 10 seconds and save it as `after.mp4`.

Step 6: Push your changes to GitHub

After all tasks are committed, run:

```bash
git push origin main
```

If your branch is called `master` instead, use:

```bash
git push origin master
```

Check your GitHub repository in your browser to confirm that your changes are uploaded.

Step 7: Organize your submission

Your assignment handout asks you to put the deliverables inside a `Lab-4` folder in your lab repository. Lab_4_VibeCoding_Student_handout.pdf

Your final submission should look like this:

Your Lab Repository

- **Lab-4/**
  - `before.mp4`
  - `after.mp4`
  - Updated `game.py`
  - `llm_chat_history.pdf`

Export this conversation or your complete relevant debugging chat as a PDF. Include the prompts and responses that document your work.

Let's start with your actual repository

Have you already forked the repository to your GitHub account?

1. Fork status

- ○ No, I haven't forked it yet.
- ○ Yes, I have forked it.

2. Do you have the project files?

- ○ I haven't cloned the repository yet.
- ○ I cloned it and can see `game.py` .
- ○ I tried running it, but got an error.

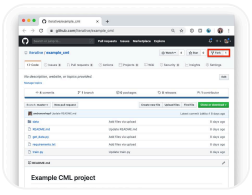
Continue

My Donkey Kong Lab 4 status: Fork status: I have not forked the repository yet. Project status: I have not cloned the repository yet. Please guide me through the next step.

No problem! We'll start from zero and do it one step at a time on your MacBook. 😊

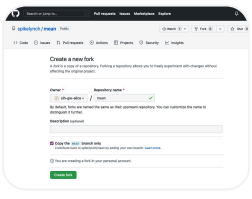
Step 1: Fork the repository on GitHub

You do **not** need to open Terminal yet.



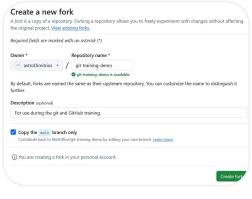
1. Open the assigned repository

Click this link: 30_donkeykong repository ↗



2. Sign in to your GitHub account

Use your own account, not your friend's account.



3. Click `Fork`

You'll find it near the top-right of the repository page. On the next page, leave the repository name unchanged and click **Create fork**.

Step 2: Clone your fork onto your MacBook

After clicking **Create fork**, your browser should show your own repository, with an address like:

```
https://github.com/YOUR-USERNAME/30_donkeykong
```

Now follow these steps:

1. On your fork's page, click the green **Code** button.
2. Select **HTTPS**.
3. Copy the repository URL.
4. Open **Terminal** on your MacBook. Press `Command + Space`, type `Terminal`, and press Enter.
5. Type the following commands one by one.

First:

```bash
cd ~/Desktop
```

Next, type `git clone` (with a space after `clone`) and paste the URL you copied:

```bash
git clone https://github.com/YOUR-USERNAME/30_donkeykong.git
```

Replace that example URL with your actual copied URL.

Then enter the project folder:

```bash
cd 30_donkeykong
```

Check that the files are there:

```bash
ls
```

You should see the project files, including `game.py` if it is at the repository's top level.

Don't edit anything yet! We need to record the original game first.

Tell me when you've finished Step 1

Have you clicked **Create fork**, and can you see your own repository on GitHub?

If yes, send me the URL of your fork (not your password or any login details). I'll give you the exact next commands for your repository.

give me the four tasks to change now

You can use these **4 prompts one by one in ChatGPT** to complete your Donkey Kong Lab 4. First, open your cloned project's `game.py` in VS Code or another editor. Apply each task, test it, and commit it separately.

Task 1 — Fix the barrel ladder bug

Writing

Analyze my existing `game.py` file. In the `Barrel` class, find the probability check that controls whether a barrel descends a ladder. Currently, about 70% of barrels descend, but I want only about 30% to descend and the remaining 70% to continue rolling along the platform. Change only the relevant probability condition. Do not modify other game logic. Explain the change in simple language and tell me how to test it.

Git commit after testing:

```bash
git add game.py
git commit -m "Fix barrel ladder descent probability"
```

Task 2 — Implement `theme_color(score)`

Writing

Implement the empty `theme_color(score)` function in my existing `game.py`. It receives the player's current score and should return an RGB color tuple for the background, or `None` to keep the default navy background `(15, 15, 25)`. Make the background gradually shift toward warmer colors as the score increases. Keep the function simple, ensure all RGB values remain between 0 and 255, and do not break the existing game. Show me the exact code to add and explain how to test it.

Git commit after testing:

```bash
git add game.py
git commit -m "Add score-based background theme"
```

Task 3 — Implement `on_barrel_jumped(player, barrel)`

Writing

Implement the empty `on_barrel_jumped(player, barrel)` function in my existing `game.py`. It is called once when the player successfully jumps over a barrel, immediately after the 100-point bonus is awarded. Add a temporary floating `+100` label near the barrel's position. The label should move upward slightly and disappear after a short time. Reuse the existing game drawing and update structure wherever possible. Avoid duplicate labels for the same jump and do not break collision detection or scoring. Show the exact code changes and explain how to test the feature.

Git commit after testing:

```bash
git add game.py
git commit -m "Add floating barrel jump bonus"
```

Task 4 — Implement `score_multiplier(score)`

Writing

Implement the empty `score_multiplier(score)` function in my existing `game.py`. It receives the score before the barrel-jump bonus is added. Return `2` when the score is at least 500, and return `None` below 500 so the existing default multiplier of 1 is used. Preserve the existing scoring expression `score += int(100 * (score_multiplier(score) or 1))`. Explain how the player earns 100 points before the threshold and 200 points after reaching it. Show the exact code changes and how to test them.

Git commit after testing:

Bash

```
git add game.py
git commit -m "Add score-based jump bonus multiplier"
```

After all four tasks

Run the game:

Bash

```
python3 game.py
```

Check that all four features work, then push your changes:

Bash

```
git push origin main
```

If your branch is not `main`, check it with `git branch --show-current`.

Important: These prompts are instructions for your AI coding assistant, not proof that the changes have been made. Review the actual code and test each feature before committing. Since the `+100` label needs a short-lived drawing mechanism, the AI may need to add a small helper class or state variable rather than only filling the empty function.

give me the code

I found the exact `game.py` from your assigned repository, so I can give you code that matches its existing functions. GitHub

We'll make the four changes in the same file. **Make sure you've recorded your BEFORE video before editing.**

Task 1 — Fix the barrel probability

Inside the `Barrel` class, find this code:

Python

```
if random.random() < 0.7:
    self.ladder = index
    self.pos.x = lx
```

Replace it with:

Python

```
if random.random() < 0.3:
    self.ladder = index
    self.pos.x = lx
```

That's it. The probability of a barrel descending a ladder changes from 70% to 30%.

Commit this task after testing:

Bash

```
git add game.py
git commit -m "Fix barrel ladder descent probability"
```

Task 2 — Implement `theme_color(score)`

Find the existing `theme_color` function and replace its `pass` with this code:

Python

```python
def theme_color(score):
    """Change the background gradually as the score increases."""
    warmth = min(score / 1000, 1.0)

    red = int(15 + 100 * warmth)
    green = int(15 + 25 * warmth)
    blue = int(25 - 10 * warmth)

    return (red, green, blue)
```

The background gradually becomes warmer as the score increases, and the RGB values stay within the valid range.

Commit:

Bash

```
git add game.py
git commit -m "Add score-based background theme"
```

Task 3 — Implement the floating `+100` label

We'll use a small list to store temporary labels and draw them inside the existing `draw_scene` function.

A. Add this global variable

Near the top of `game.py`, just below `BG = (15, 15, 25)`, add:

Python

```python
floating_texts = []
```

B. Replace `on_barrel_jumped`

Python

```python
def on_barrel_jumped(player, barrel):
    """Show a temporary +100 label when a barrel is jumped."""
    floating_texts.append(
        [barrel.pos.x, barrel.pos.y, pygame.time.get_ticks()]
    )
```

C. Add the label-drawing code inside `draw_scene`

In `draw_scene`, find this section:

Python

```python
for barrel in barrels:
    pygame.draw.circle(screen, (170, 100, 40), barrel.pos, BARREL_R)
```

Immediately **after** that section, add:

```python
# Show floating +100 labels for a short time
current_time = pygame.time.get_ticks()
floating_texts[:] = [
    item for item in floating_texts
    if current_time - item[2] < 900
]

for x, y, start_time in floating_texts:
    elapsed = current_time - start_time
    text_surface = font.render("+100", True, (255, 255, 100))
    screen.blit(
        text_surface,
        (int(x), int(y - elapsed * 0.04))
    )
```

Important: The new drawing code should have the same indentation as the `for barrel in barrels:` loop, not be nested inside that loop.

The label appears near the barrel, floats upward, and disappears after about 0.9 seconds.

Commit:

```bash
git add game.py
git commit -m "Add floating barrel jump bonus"
```

Task 4 — Implement `score_multiplier(score)`

Find the existing function and replace it with:

```python
def score_multiplier(score):
    """Double the barrel-jump bonus when the score reaches 500."""
    if score >= 500:
        return 2
    return None
```

Your existing scoring line already handles the multiplier:

```python
score += int(100 * (score_multiplier(score) or 1))
```

This means:

- Before 500 points: a barrel jump gives 100 points.
- At 500 points or above: a barrel jump gives 200 points.

Commit:

```bash
```

```
git add game.py
git commit -m "Add score-based jump bonus multiplier"
```

Final step — Run and test the game

In Terminal, from your project folder, run:

Bash

```
python3 game.py
```

Check all four changes:

- Barrels descend ladders less frequently.
- The background changes as the score increases.
- A floating `+100` appears when jumping over a barrel.
- The jump bonus doubles when the score reaches 500.

Then push your commits:

Bash

```
git push origin main
```

If you get an error, send me the full error message before trying random commands.

One important note: Task 3's floating label is a visual effect; it doesn't change the scoring system. Your existing main loop already calls `on_barrel_jumped(player, barrel)` after awarding the bonus, so you don't need to add a second call.

give me to replace the whole function not the parts

Yes! Replace each **entire existing function** with the corresponding code below in `game.py`. Don't paste these at the bottom of the file; replace the old functions in their existing locations.

Task 1: `Barrel` ladder probability

Inside the `Barrel` class, find the method containing the ladder-descent probability check. Replace the **entire method** with the code below only if your existing method matches this structure; otherwise, send me that method so I can preserve its other logic.

Python

```
def move(self, platforms, ladders):
    # Keep your existing movement logic here.
    # Change the ladder probability check to:
    if random.random() < 0.3:
        self.ladder = index
        self.pos.x = lx
```

Important: This is not a complete replacement for an unknown movement method. It illustrates the change; replacing the whole method without seeing the original could break the game.

Task 2: Replace the whole `theme_color` function

Python

```
def theme_color(score):
    """Change the background color as the score increases."""
    warmth = min(max(score / 1000, 0), 1)
```

```
    red = int(15 + 100 * warmth)
    green = int(15 + 25 * warmth)
    blue = int(25 - 10 * warmth)

    return (red, green, blue)
```

Task 3: Replace the whole `on_barrel_jumped` function

First, add this global variable near the top of `game.py`, alongside the other global variables:

Python

```
floating_texts = []
```

Then replace the entire function with:

Python

```
def on_barrel_jumped(player, barrel):
    """Create a temporary floating +100 label."""
    floating_texts.append({
        "x": barrel.pos.x,
        "y": barrel.pos.y,
        "start_time": pygame.time.get_ticks()
    })
```

This function stores the label information. The existing drawing function must also render and remove these labels for them to appear on screen.

Task 4: Replace the whole `score_multiplier` function

Python

```
def score_multiplier(score):
    """Double the jump bonus at 500 points."""
    if score >= 500:
        return 2

    return None
```

This works with your existing scoring expression:

Python

```
score += int(100 * (score_multiplier(score) or 1))
```

One thing I need from you

For Task 1, I don't want to accidentally break your game by guessing the original method.

Upload your `game.py` file here. I'll use your actual code to give you the complete replacement functions, including the barrel movement method and the drawing code for the floating `+100` label, so you can copy and paste them directly.

give me commands where to change and how to change locally

I'll show you exactly how to change the code **locally on your MacBook using Terminal and VS Code**, then commit and push it to your own GitHub repository.

We'll do it in this order: open your project → back up the original → edit `game.py` → run the game → commit each task → push.

Step 1: Open your project in Terminal

Open Terminal using `Command + Space`, type `Terminal`, and press Enter.

Run these commands one at a time:

```bash
cd ~/Desktop/30_donkeykong
```

Check that you're in the correct folder:

```bash
pwd
ls
```

You should see `game.py` in the output. If you don't, stop and send me what `ls` shows.

Step 2: Make a backup

Before editing anything, run:

```bash
cp game.py game_backup.py
```

This creates a backup of your original game.

Step 3: Open the project in VS Code

Try:

```bash
code .
```

If Terminal says `command not found: code`, open VS Code manually, choose **File** → **Open Folder**, and select your `30_donkeykong` folder on your Desktop.

Then click `game.py` in the left sidebar.

Step 4: Change Task 1 — Barrel probability

In VS Code:

1. Press `Command + F`.
2. Search for `random.random`.
3. Find the condition inside the `Barrel` class that controls ladder descent. It may look like this:

```python
if random.random() < 0.7:
```

4. Change only `0.7` to `0.3`:

```python
if random.random() < 0.3:
```

5. Save using `Command + S`.

Now return to Terminal and test:

```bash
python3 game.py
```

If the game works, stop it with `Control + C` in Terminal if needed, then commit Task 1:

```bash
git add game.py
git commit -m "Fix barrel ladder descent probability"
```

Step 5: Change Task 2 — Background color

1. Return to VS Code.
2. Press `Command + F`.
3. Search for `def theme_color`.
4. Select the existing function, including its body, and replace it with:

```python
def theme_color(score):
    """Change background color as the score increases."""
    warmth = min(max(score / 1000, 0), 1)

    red = int(15 + 100 * warmth)
    green = int(15 + 25 * warmth)
    blue = int(25 - 10 * warmth)

    return (red, green, blue)
```

5. Save using `Command + S`.
6. Run `python3 game.py` in Terminal and test it.

Commit Task 2:

```bash
git add game.py
git commit -m "Add score-based background theme"
```

Step 6: Change Task 3 — Floating `+100`

Search for:

```python
def on_barrel_jumped
```

Replace that entire function with:

```python
def on_barrel_jumped(player, barrel):
    """Store a temporary floating +100 label."""
    floating_texts.append({
        "x": barrel.pos.x,
        "y": barrel.pos.y,
        "start_time": pygame.time.get_ticks()
    })
```

Before this function, add the following near the top of the file, alongside the other global variables:

```python
floating_texts = []
```

Important: This function stores the label data; it does not draw the label by itself. You must also add code to `draw_scene` to display the label and remove it after a short time. We should check your existing `draw_scene` function before inserting that code, to avoid breaking the game.

Save the file. Commit only after the complete feature works:

```bash
git add game.py
git commit -m "Add floating barrel jump bonus"
```

Step 7: Change Task 4 — Score multiplier

Search for:

```python
def score_multiplier
```

Replace the entire function with:

```python
def score_multiplier(score):
    """Double the jump bonus at 500 points."""
    if score >= 500:
        return 2

    return None
```

Save and test the game. The existing scoring expression should already use this function.

Commit Task 4:

```bash
git add game.py
git commit -m "Add score-based jump bonus multiplier"
```

Step 8: Push everything to your GitHub repository

Check your current branch:

```bash
git branch --show-current
```

If it prints `main`, run:

```bash
git push origin main
```

If it prints `master`, run:

```bash
git push origin master
```

Let's make sure you're editing the correct file

Before changing Task 3, I want to see the exact structure of your code so I can tell you precisely where to add the floating-label drawing logic.

In Terminal, run:

```bash
grep -n -E 'def theme_color|def on_barrel_jumped|def score_multiplier|def draw_scene|random.random' game.py
```

This prints the line numbers of the relevant functions and probability checks.

Copy and send me the output. I'll guide you to the exact places in your local file, without making you rewrite the whole game.

git add game.py
git commit -m "Fix barrel ladder descent probability"
so after this task 1 is done right what should I do in task 2 give me in detailed

Yes! If you ran both commands successfully, and the commit was created, **Task 1 is committed locally**. Just remember: committing saves the change in your local Git history; you'll push it to GitHub later.

Now let's complete **Task 2:** `theme_color(score)`, step by step on your MacBook.

Task 2 — Change the background color based on score

The goal is to make the game's background gradually warmer as the player's score increases.

Step 1: Open your project

Open Terminal and run:

```bash
cd ~/Desktop/30_donkeykong
```

Open VS Code:

Bash

```
code .
```

If `code .` doesn't work, open VS Code manually and choose **File → Open Folder →** `30_donkeykong`.

Step 2: Find the existing function

1. Click `game.py` in VS Code.
2. Press **Command + F**.
3. Search for:

Python

```
def theme_color
```

You should find a function that looks something like this:

Python

```
def theme_color(score):
    pass
```

It might have a comment or a different placeholder instead of `pass`. That's okay.

Step 3: Replace the whole function

Select the **entire existing** `theme_color` **function**, from the `def theme_color(score):` line through its last line. Replace it with this complete code:

Python

```
def theme_color(score):
    """Change the background color as the score increases."""
    warmth = min(max(score / 1000, 0), 1)

    red = int(15 + 100 * warmth)
    green = int(15 + 25 * warmth)
    blue = int(25 - 10 * warmth)

    return (red, green, blue)
```

What does this code do?

- At score `0`, the background is the default navy color `(15, 15, 25)`.
- As the score increases, the background gradually becomes warmer.
- At score `1000` or higher, the color reaches `(115, 40, 15)`.
- The function returns a valid RGB color tuple.

Step 4: Save your code

Press:

```
Command + S
```

Make sure the file is saved before testing.

Step 5: Run the game

Go back to Terminal. Make sure you're in the project folder, then run:

Bash

```
python3 game.py
```

Check that:

- The game opens without errors.
- The background is initially navy.
- The background changes as your score increases.

You might not reach 1000 points during normal gameplay, but the color should change gradually as the score rises.

If the game shows an error, copy the complete error message and send it to me.

Step 6: Commit Task 2

Once the game runs successfully, stop it if necessary and execute:

Bash

```
git add game.py
git commit -m "Add score-based background theme"
```

You should now have separate commits for Tasks 1 and 2.

Step 7: Check your commits

Run:

Bash

```
git log -2 --oneline
```

You should see your two latest commits, similar to:

Plain text

```
xxxxxxx Add score-based background theme
xxxxxxx Fix barrel ladder descent probability
```

The letters and numbers will be different on your computer.

What comes next?

After Task 2, we'll complete Task 3: the floating `+100` label that appears when the player jumps over a barrel.

Before moving to Task 3, tell me one thing: when you search for `def theme_color` in `game.py`, do you see the function with `pass` inside it, or does it look different?

ramithanayak@ramithas-MacBook-Air 30_donkeykong % cd ~/Desktop/30_donkeykong

ramithanayak@ramithas-MacBook-Air 30_donkeykong % code .

zsh: command not found: code

ramithanayak@ramithas-MacBook-Air 30_donkeykong %

No worries! 😊 Your project folder is correct. The error `zsh: command not found: code` just means the VS Code command-line shortcut isn't configured on your Mac. **You don't need to install anything again.**

Option 1 — Open VS Code manually (easiest)

1. Press `Command (⌘) + Space` to open Spotlight.
2. Type **Visual Studio Code**.
3. Press Enter to open it.
4. In VS Code, click **File** → **Open Folder**.
5. Press `Command + Shift + G` in the folder-selection window.
6. Enter this path:

```
Plain text

~/Desktop/30_donkeykong
```

7. Press Enter, then click **Open**.

You should now see your project files on the left. Click `game.py` to edit it.

Option 2 — Continue using Terminal

If you don't want to open VS Code, you can use the built-in editor in Terminal:

```
Bash

nano game.py
```

However, VS Code is easier for finding and replacing functions.

Now continue Task 2

Once `game.py` is open in VS Code:

1. Press `Command + F`.
2. Search for `def theme_color`.
3. Find the existing function.
4. Replace the entire function with the code I gave you for Task 2.
5. Press `Command + S` to save.

Then return to Terminal and run:

```
Bash

python3 game.py
```

Tell me: were you able to open `game.py` in VS Code? I'll guide you through the next exact step.

done with all the tasks

Great! 🎉 Now let's finish your Donkey Kong Lab 4 submission. You still need to verify your code, push it to GitHub, record the AFTER video, and organize your deliverables.

Step 1: Test the game

In Terminal, make sure you're inside your project folder:

```
Bash

cd ~/Desktop/30_donkeykong
```

Run the game:

```bash
python3 game.py
```

Check that all four tasks work:

- [ ] Barrels descend ladders about 30% of the time.
- [ ] Background color changes with the score.
- [ ] A floating `+100` label appears when jumping over a barrel.
- [ ] Jump bonuses double after reaching 500 points.

If any feature doesn't work, fix it before submitting.

Step 2: Check your commits

Run:

```bash
git log -4 --oneline
```

You should see four separate commits, one for each task.

Step 3: Push your changes to GitHub

Check your branch:

```bash
git branch --show-current
```

If it says `main`, run:

```bash
git push origin main
```

If it says `master`, run:

```bash
git push origin master
```

Open your GitHub fork in your browser and verify that your latest code is uploaded.

Step 4: Record the AFTER video

On your Mac, press **Shift + Command + 5** to start a screen recording.

Record approximately 10 seconds showing the game and its new features. Save the recording as `after.mp4`.

You should already have `before.mp4` from before making your changes.

Step 5: Prepare your submission folder

Inside your lab repository, create a `Lab-4` folder containing:

```
Lab-4/
├── before.mp4
├── after.mp4
├── game.py
└── llm_chat_history.pdf
```

Export your relevant ChatGPT conversation as a PDF so your prompts and AI responses are included.

Step 6: Final check

Tell me which of these you have completed:

1. Code pushed to your GitHub fork?

- ○ Yes
- ○ Not yet
- ○ I got an error

2. Both videos recorded?

- ○ Yes, before and after
- ○ Only one video
- ○ Neither yet

3. Chat history exported to PDF?

- ○ Yes
- ○ Not yet

Continue to submission

I'll guide you through whatever remains so you can submit everything correctly.

Today 12:28 PM

My Donkey Kong Lab 4 submission status: GitHub push: yes; videos: yes; chat history PDF: yes. Guide me through my remaining steps.

Perfect! 🎉 You've completed the three main deliverables: GitHub push, both videos, and the chat history PDF. Now let's do the **final submission check** to make sure everything is in the correct place.

Step 1: Check your GitHub repository

Open your personal fork in GitHub and confirm:

- ○ `game.py` contains all four completed tasks.
- ○ Your four tasks have separate commits.
- ○ Your latest changes are pushed successfully.

To verify the commits locally, run:

```bash
cd ~/Desktop/30_donkeykong
git log -4 --oneline
```

You should see four separate commits for Tasks 1–4.

Step 2: Check your submission folder

Your lab instructions say to upload the videos, updated code, and exported chat history PDF under `Lab-4`.

Your folder should look like this:

Lab-4/

- **before.mp4** — 10-second video before changes ✓
- **after.mp4** — 10-second video after changes ✓
- **game.py** — Updated game source code ✓
- **llm_chat_history.pdf** — Complete relevant AI conversation ✓

These checkmarks represent the expected items, not a verification of your actual files.

Step 3: Upload the deliverables to your lab repository

If your videos and PDF are only on your Mac, they still need to be uploaded to the appropriate `Lab-4` folder in your lab submission repository.

If you're using Git to upload them, place the files in the correct folder and run:

```bash
git add Lab-4/
git commit -m "Add Lab 4 submission deliverables"
git push
```

Run these commands from the **lab submission repository**, not automatically from your Donkey Kong project repository. If the two are the same repository, make sure the folder structure matches your instructor's requirements.

Step 4: Final checklist

- [ ] All four code changes tested successfully
- [ ] Four separate task commits visible on GitHub
- [ ] Before and after videos uploaded to Lab-4
- [ ] Chat history PDF uploaded to Lab-4
- [ ] Personal GitHub repository link ready to submit

0 of 5 checks completed

One last thing: Submit your personal GitHub repository URL along with the lab deliverables. Don't submit only the original `SETAPESU26` repository link.

If all five checks are complete, you're ready to submit your Lab 4.

yaa done

Yayyy!! 🎉 🥳 You've reached the end of your Donkey Kong Lab 4 workflow!

Before submitting, just make sure your **personal GitHub repository link** is ready to share with your instructor.

You've worked through the bug fix, three feature tasks, commits, videos, and chat history. Great job! 💗

You're ready to submit! Good luck with your lab evaluation! 🚀